

Neural networks estimation of the density in molecular clouds : Benchmark on Orion B dense cores

Z. Ribeiro^{1*} and P. Hily-Blant¹

Univ. Grenoble Alpes, CNRS, IPAG, 38000 Grenoble, France

Received September XX, 20XX

ABSTRACT

Context. /

Aims. This work explores the use of different neural network architectures to estimate the mass-weighted average volume density along the line of sight in molecular clouds, as well as the average density of dense cores. We aim to provide a full benchmark of these architectures, highlighting their advantages and limitations in this physical context.

Methods. We use synthetic datasets generated from magnetohydrodynamic simulations from which column and average volume density maps were derived. These datasets allows the training of different neural networks: a CNN (UNet) and three conditioned generative ones: cINNs, DDPMs and Flow Matching. We apply differents numericals and physicals metrics to test the robustness, confidence and limitations of each method.

Results. /

Conclusions. /

Key words. ISM: clouds – ISM: structures – stars: formation – machine learning – neural networks – mass function

1. Introduction

2. Methods

In this work, because our goal is to study dense structures in molecular clouds, such as prestellar cores, we choose to estimate, using neural networks, the mass-weighted average volume density along the line of sight:

$$\langle n_H \rangle_m(x, y) = \frac{\int_0^L n_H(x, y, z) m(x, y, z) dz}{\int_0^L m(x, y, z) dz}, \quad (1)$$

from the column density map derived from dust emission:

$$N_H(x, y) = \int_0^L n_H(x, y, z) dz, \quad (2)$$

where x and y are the plane-of-sky positions, z is the position in line of sight axis, L the cloud length, n_H the hydrogen volume density, $m(x, y, z) = n_H(x, y, z) \mu m_H$.

Thus, our benchmark focuses on a $2D \rightarrow 2D$ learning problem. Note also that the mass-weighted average density can be converted into the average density of dense cores using a small correction (see Appendix A).

Except for making the simulations, all steps, from creating the datasets and training neural networks to applying them to observations, are performed using our code POLARIScore¹, which includes our implementations of network architectures using PyTorch (Paszke et al. 2019).

2.1. MHD Simulations and Datasets

Neural networks need to be trained on synthetic data, where they can learn the relevant correlations and underlying physics. For segmentation tasks, such training data can consist of human-generated or algorithm-generated labels on observations (see, for example, filament-identification studies (Alina et al. 2022)). However, in our task of estimating a physical quantity in molecular clouds, we do not have access to true observational values. This is where simulations become essential, since they provide full access to all required physical quantities.

The properties of the simulations used to train the neural network are therefore extremely important and directly influence the network's performance. **talk about properties of the cloud Orion B**

These datasets are created using magnetohydrodynamical (MHD) simulations of a star-forming molecular cloud: *ORION* (Ntormousi & Hennebelle 2019), publicly available through the GALACTICA² database and shown in Figure 1.

These simulations are isothermals, model MHD turbulence and have a size of 66.1 pc with an initial resolution of 0.01 pc, reaching a few hundred AU using an AMR grid. The simulations were performed using RAMSES (Teyssier 2002), with self-gravity applied, star-formation treated using Lagrangian sink particles and ideal MHD resolved and no chemistry.

To facilitate data manipulation, we use a 1024^3 density cube with a width of 25.8pc, centered at the simulation's midpoint, with a resolution of 0.026pc per pixel. To increase the amount of training data, projections along the three axes are used. In other words, we generate three column density images by summing the densities along different axes as shown in Figure 1.

* zack.ribeiro@univ-grenoble-alpes.fr

¹ <https://github.com/ZackRibeiro/POLARIScore>

² http://www.galactica-simulations.eu/db/STAR_FORM/ORION/

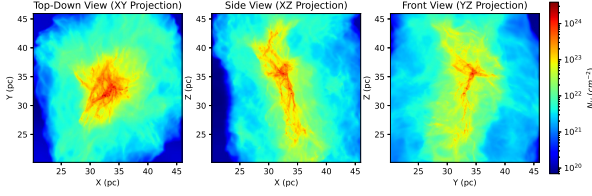


Fig. 1: Column densities for 3 axes of the same molecular cloud from the low magnetic field ORION simulation. (Ntormousi & Hennebelle 2019)

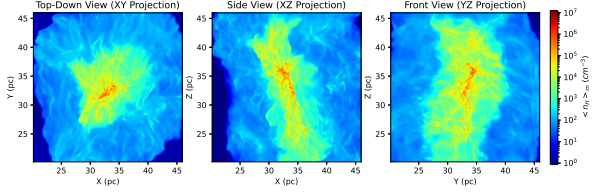


Fig. 2: Mass-weighted average volume densities for 3 axes of the same molecular cloud from the low magnetic field ORION simulation. (Ntormousi & Hennebelle 2019)

Then, images with a resolution of 128×128 pixels and a physical size of 3.33pc are randomly cropped from the three faces. An image is retained in the dataset if it meets several conditions:

- It is spatially distant enough from an already explored region, meaning it does not overlap with an area covered by an existing image.
- It does not contain a disproportionately large fraction of very low-density regions, i.e., the edges of the simulation (WNM).
- It exhibits significant density variations within the explored regions. A score is based on the smoothness of the image using Laplacian of the density, favoring regions with filaments over those with nearly homogeneous density. (See Appendix D)

The generated dataset is then randomly shuffled and split between training set (70%-80%) and validation set (30%-20%). For example, using only the simulation shown in Figure 1, we obtain a total of ≈ 100 covered regions, each represented by a pair of images: (column density, volume density)

2.2. Training

Training a neural network is a crucial step in achieving good performance. There are multiple choices that we need to make to obtain optimal results.

- We want the model, with N free parameters (weights), to converge to a global minimum of the loss function rather than getting stuck in a local well. Therefore, the choice of optimizer, which performs backward error propagation (adjusting the model's weights), is important. Here, we use ADAM (Kingma & Ba 2017).
- Loss function: A well-chosen loss function is essential for accurately representing the problem. Here, we generally use the mean squared error or minimizing the negative log-likelihood depending on the architecture.
- Learning rate: The learning rate is another critical factor, it determines the step size the optimizer takes in the loss function during each update (epoch). Ideally, it should decrease

when the loss reaches a plateau. To manage this, we use a *scheduler*.

Each epoch, i.e., learning step, the model processes all the dataset, column density images, and predicts their mass-weighted density. A way to artificially increase the size of the training batch is to apply random transformations to the images at each epoch (Yang et al. 2023), such as random rotations and random vertical or horizontal flips (no deformations). Afterward, the prediction loss is computed using the chosen loss method, and backward error propagation is performed. Then, a new epoch begins, repeating the same steps.

2.3. Machine learning

In this benchmark, we explore four different neural network architectures. We start with a robust and widely used model for image-to-image regression tasks, the UNet (Ronneberger et al. 2015; Oktay et al. 2018). We then test more recent state-of-the-art conditioned generative approaches, namely Conditional Invertible Neural Networks (cINNs) (Ardizzone et al. 2021), Diffusion Models (Ho et al. 2020), and latest Flow-Matching models (Lipman et al. 2023).

We do not include Generative Adversarial Networks (GANs) (Goodfellow et al. 2014) in this work mainly because they are known to suffer from mode collapse (Kossale et al. 2022).

2.3.1. Convolutional Neural Network

Convolutional Neural Networks (CNNs) were introduced (Fukushima (1980), LeCun et al. (1989)) as a specialized architecture for grid-like data such as images or maps. Instead of connecting every input to every neuron (as in multi-layer perceptrons), CNNs restrict connections to local receptive fields. Each neuron in a convolutional layer is connected only to a small region of the input, and the same set of learnable weights (a filter or kernel) is shared across different spatial locations. This enables CNNs to learn spatially invariant features such as edges, textures and structure morphologies.

Mathematically, for a 2D input X (e.g a column density region), the convolution operation with a filter W is given by:

$$Y_{i,j} = f\left(\sum_{m=1}^M \sum_{n=1}^N W_{m,n} \cdot X_{i+m,j+n} + b\right), \quad (3)$$

where W is the learnable convolutional kernel, X is the input feature map, Y is the output feature map, f is a nonlinear activation function such as ReLU, b is the learnable bias term.

In this work, we are using an Attention-U-Net (Ronneberger et al. (2015), Oktay et al. (2018)) architecture, shown in Figure 3. It features a symmetric encoder-decoder structure where the encoder captures high-level features of the column density map through convolution (Eq 3) and max pooling operations, while the decoder gradually reconstructs the mass-weighted density map using upsampling layers.

A key feature of U-Net is its *skip connections*, which allow direct information flow between corresponding encoder and decoder layers, preserving spatial details lost during downsampling. **small sentence on Attention blocks ?**

2.3.2. Conditional Invertible Neural Network

However, state-of-art networks used today are not deterministic architectures such as UNet, but stochastic models that sample from probability distributions.

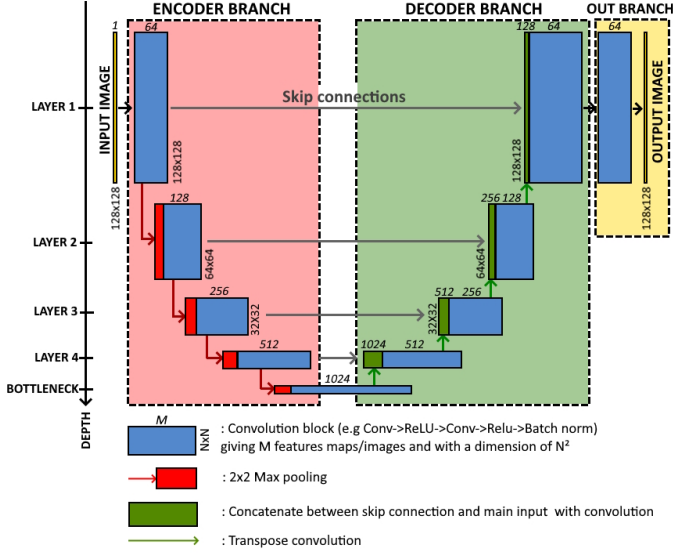


Fig. 3: U-Net architecture (Ronneberger et al. 2015). Each blue-box corresponds to a convolutional block. The number of channels is denoted on top of the box. The x-y-size is provided at the right or left edge of the box.

The goal of these networks is to sample from the conditional data distribution $p_{\text{data}}(X|c)$ where X is the target random variable (here $\langle n_H \rangle_m$) and c the conditioning information (from observable, here N_H). In our case, if the inverse problem is ill-posed, i.e if one cannot uniquely infer the mass-weighted average volume density map $\langle n_H \rangle_m(x, y)$ from the 2D column-density map $N_H(x, y)$, the generative network instead learns to sample physically plausible solutions drawn from the full posterior distribution

$$p(\langle n_H \rangle_m(x, y) | N_H(x, y)). \quad (4)$$

To make sampling tractable, we map the unknown complex distribution to a simple latent one, typically a standard multivariate Gaussian:

$$p_z(Z) = N(0, I), \quad (5)$$

where Z represents the latent variables.

The network then learns a bijective transformation f between X and Z , conditioned on c :

$$Z = f(X, c), \quad X = f^{-1}(Z, c) \quad (6)$$

The approach used in conditional invertible neural networks (cINN, Ardizzone et al. (2021)) is to train the forward direction

$$f : X \rightarrow Z \quad (7)$$

such that the pushforward of the data distribution becomes Gaussian:

$$f_*(p_{\text{data}}(X|c)) = p_z(Z). \quad (8)$$

This is achieved through normalizing flows (Rezende & Mohamed 2016), which transform a complex distribution into a simple one via a sequence of invertible transformations: $f_1, \dots, f_K$:

$$Z = f_K \cdot f_{K-1} \cdot \dots \cdot f_1(X, c). \quad (9)$$

For each invertible transformation f_k , the change-of-variable formula gives the exact likelihood:

$$p_{\text{data}}(X|c) = p_z(f(X, c)) \left| \det \frac{\partial f(X, c)}{\partial X} \right|. \quad (10)$$

During training, the cINN maximizes the log-likelihood of the data under this model. At inference time, we draw $Z \sim N(0, I)$ and obtain samples of the physical solution via the inverse mapping: $X = f^{-1}(Z|c)$.

To ensure reasonable computation time, each transformation f_k in the flow must be easily invertible, and its Jacobian determinant must be efficient to compute. cINNs achieve this by using conditional coupling blocks (CCBs). Let the input of a coupling block be split (in features channels) as $u \rightleftharpoons (u_1, u_2)$, and the output be $v \rightleftharpoons (v_1, v_2)$. A typical forward transformation of a CCB is:

$$\begin{cases} v_1 = u_1 \odot \exp(s_1(u_2, c)) + t_1(u_2, c), \\ v_2 = u_2 \odot \exp(s_2(v_1, c)) + t_2(v_1, c), \end{cases} \quad (11)$$

where s_i, t_i are scale and translation functions implemented by sub-networks (e.g MLPs). These sub-networks do not need to be invertible because they are only evaluated in the forward pass; the coupling block itself remains analytically invertible. The inverse transformation is obtained by algebraic rearrangement of Equation 11:

$$\begin{cases} u_2 = (v_2 - t_2(v_1)) \odot \exp(-s_2(v_1)) \\ u_1 = (v_1 - t_1(u_2)) \odot \exp(-s_1(u_2)) \end{cases} \quad (12)$$

Because each coupling block is triangular in structure, its Jacobian is also triangular. Thus the log-determinant of the Jacobian is simply the sum of the scaling terms:

$$\log |\det J| = \sum s_1(u_2, c) + \sum s_2(v_1, c), \quad (13)$$

where the sums run over features and spatial dimensions. This makes CCBs computationally efficient while retaining exact invertibility and tractable likelihood.

Some words on architecture with figure (Encoder, Haar wavelet downsampling)

2.3.3. Denoising Diffusion Probabilistic Model

Denoising Diffusion Probabilistic Models (DDPMs, Ho et al. (2020)) generate samples by gradually transforming white noise, i.e a sample from the latent Gaussian prior $p_z = N(0, I)$, into a data sample $x_0 \sim p_{\text{data}}$. This is achieved through a sequence of learned denoising steps parameterized by a neural network, typically a UNet.

The forward diffusion process progressively destroys structure by adding Gaussian noise with a predefined variance schedule β_t . In the continuous-time limit, this process is described by the stochastic differential equation (Itô SDE):

$$dx = f(x, t)dt + g(t)dW_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)}dW_t, \quad (14)$$

where W_t is a Wiener process. Here:

- $g(t) = \sqrt{\beta(t)}$ is the diffusion strength,
- $f(x, t) = -\frac{1}{2}\beta(t)x$ is the drift term,
- $t \in [0, T]$ parametrized the diffusion trajectory.

This SDE is a variant of the Ornstein-Uhlenbeck process, whose solutions converge to a Gaussian distribution (Uhlenbeck & Ornstein 1930). The process is variance-preserving, since as $t \rightarrow \infty$, $x_t \sim N(0, I)$.

To generate data, DDPMs simulate the reverse-time SDE:

$$dx = [f(x, t) - g(t)^2 \nabla_{x_t} \log p_t(x_t)]dt + g(t)d\bar{W}_t, \quad (15)$$

where $\bar{W}_t$ is a Wiener process running backward in time. The quantity

$$\nabla_{x_t} \log p_t(x_t) \quad (16)$$

is the score function, i.e the gradient of the log-density of the noisy data at time t . Because the exact score is intractable, the network learns an approximation $s_\theta(x_t, t) \approx \nabla_{x_t} \log p_t(x_t)$.

In DDPM parameterization, the neural network instead predicts the noise

$$s_\theta(x_t, t) \approx \frac{\epsilon_\theta(x_t, t)}{\sigma_t}, \quad (17)$$

where $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$ is the standard deviation of the forward noising kernel.

The forward noising process has the closed-form solution:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim N(0, I), \quad (18)$$

where $\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$ encodes the cumulative noise schedule.

Thus, training a DDPM consists of learning to predict the injected noise ϵ , which in turn provides the score needed to solve the reverse SDE and sample realistic data from pure Gaussian noise.

In the conditional setting, the observable N_H is incorporated by concatenating it with the noisy sample x_t at the input of the UNet, ensuring that each denoising step is conditioned on the available physical information.

Some words on architecture with figure (Encoder, Noise scheduling)

Salimans & Ho (2022)

2.3.4. Flow matching

$$\begin{cases} \frac{d}{dt} \varphi_t(x_0) = u_t(\varphi_t(x_0)), \\ \varphi_0(x_0) = x_0, \end{cases} \quad (19)$$

2.4. Application

3. Results

3.1. Benchmark on simulation data

Errors evaluated on the validation data (subset of the simulation):

- **Mean Squared Error (MSE):** Standard regression metric.
- **Spatial error maps:** Residual maps used to assess whether errors are spatially random or, for example, systematically higher in filamentary regions, indicating difficulty in predicting filaments.
- **Accuracy:** Fraction of pixels with an absolute error below a given threshold (e.g., 90% of pixels within 0.5 dex (in logspace)).
- **Residual curves:** Used to diagnose whether the network underestimates high densities or low densities.

- **Inference time:** Number of predictions per second, i.e., whether a full observation can be processed in minutes or hours.
- **Model complexity:** Number of trainable parameters, allowing us to compare architectures with similar capacity.
- **Robustness to training-set variations:** Sensitivity of model performance to changes in the composition of the training set (see end of Appendix D).
- **Required training-set size:** For example, UNet reaches good performance with relatively few regions, whereas cINNs typically require larger datasets.
- **Training stability:** For instance, UNet can be trained reliably under a wide range of settings, while cINNs require more careful choices of training parameters for stable optimization.

3.2. Tests on Orion B

- **Density PDFs:** Comparison of the probability distribution functions of the predicted densities (but how?).
- **Fractal dimension:** Using the relation $P = A^{D/2}$, where P is the perimeter and A the area, we test whether the structures follow a power-law scaling, implying a constant fractal dimension D across spatial scales.
- **Typical size of structures (?):** Evaluate characteristic spatial scales (e.g., whether filament widths cluster around ~ 0.1 pc).
- **Comparison with the core catalog of Könyves et al.:** Differences between predicted core densities and catalogued values.
- **Core morphology:** Visual inspection to determine whether predicted cores are embedded within density clumps or appear isolated (hierarchical picture).
- **Robustness to resolution changes:** Evaluate how the predictions vary when the observational resolution is modified, using both the validation set and real observations.
- **$N_H(n_H)$ relation (dense cores only):** Fit the predicted column-volume density of dense cores relation to infer plausible temperatures or sound speeds.
- **Dense Core Mass Function (DCMF):** Agreement with Könyves et al. regarding peak position, width, low-mass cut-off, and high-mass power-law slope.

3.3. Summary

Table: Models vs Metrics

4. Discussions

Other methods to find mass-weighted density : Gaches & Grudić (2024), Orkisz & Kainulainen (2024).

5. Conclusions

References

- Alina, D., Shomanov, A., & Baimukhametova, S. 2022, IEEE Access, 10, 74472, arXiv:2205.00683 [astro-ph]
Ardizzone, L., Kruse, J., Lüth, C., et al. 2021, arXiv:2105.02104 [cs]
Fukushima, K. 1980, Biological Cybernetics, 36, 193
Gaches, B. A. L. & Grudić, M. Y. 2024, version Number: 1
Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., et al. 2014, version Number: 1
Ho, J., Jain, A., & Abbeel, P. 2020, version Number: 2
Kingma, D. P. & Ba, J. 2017, arXiv:1412.6980 [cs]
Kossale, Y., Airaj, M., & Darouchi, A. 2022, in 2022 8th International Conference on Optimization and Applications (ICOA) (Genoa, Italy: IEEE), 1–6

- Könyves, V., André, P., Arzoumanian, D., et al. 2020, *Astronomy & Astrophysics*, 635, A34
- LeCun, Y., Boser, B., Denker, J. S., et al. 1989, *Neural Computation*, 1, 541
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. 2023, arXiv:2210.02747 [cs]
- Ntormousi, E. & Hennebelle, P. 2019, *Astronomy & Astrophysics*, 625, A82
- Oktaç, O., Schlemper, J., Folgoc, L. L., et al. 2018, arXiv:1804.03999 [cs]
- 310 Orkisz, J. H. & Kainulainen, J. 2024, version Number: 1
- Paszke, A., Gross, S., Massa, F., et al. 2019, arXiv:1912.01703 [cs]
- Rezende, D. J. & Mohamed, S. 2016, arXiv:1505.05770 [stat]
- Ronneberger, O., Fischer, P., & Brox, T. 2015, arXiv:1505.04597 [cs]
- Salimans, T. & Ho, J. 2022, arXiv:2202.00512 [cs]
- Teyssier, R. 2002, *Astronomy & Astrophysics*, 385, 337
- Uhlenbeck, G. E. & Ornstein, L. S. 1930, *Physical Review*, 36, 823
- Yang, S., Xiao, W., Zhang, M., et al. 2023, arXiv:2204.08610 [cs]

Appendix A: From mass-weighted averages to core densities

In this work, the neural networks aim to predict the mass-weighted average volume density:

$$\langle n_H \rangle_m(x, y) = \frac{\int_0^L n_H(x, y, z) m(x, y, z) dz}{\int_0^L m(x, y, z) dz}, \quad (\text{A.1})$$

where x and y are the plane-of-sky positions, z is the position in line of sight axis, L the cloud length, n_H the hydrogen volume density, $m(x, y, z) = n_H(x, y, z) \mu m_H$.

If we discretize the density along the l.o.s into M components with constant volume density in it (e.g for a datacube simulation, M corresponds to the number of cells), then Eq A.1 becomes

$$\langle n_H \rangle_m = \frac{\sum_{i=1}^M n_{H,i}^2 \times L_i \mu m_H}{\sum_{i=1}^M n_{H,i} \times L_i \mu m_H} = \frac{\sum_{i=1}^M n_{H,i}^2 L_i}{\sum_{i=1}^M n_{H,i} L_i}, \quad (\text{A.2})$$

where L_i is the length of component i . This quantity is chosen mainly because this weighted average density is close to the average density of dense structures along the l.o.s such as dense cores.

However, when a dense core is embedded by a very diffuse region (i.e outside filamentary structures), a correction is required to convert the mass-weighted average density $\langle n_H \rangle_m$ into the true average volume density of the dense core $n_{H,c}$. To simplify notation, we denote the average core density simply by n_c and volume densities without the H .

Consider two components along the line of sight ($M = 2$):

- A diffuse medium with length L_d and with constant density n_d .
- The dense core with length $L_c = 2r_c$ where r_c is the core radius, and with constant density n_c .

Equation A.2 then reduces to

$$\langle n_H \rangle_m = \frac{n_c^2 L_c + n_d^2 L_d}{n_c L_c + n_d L_d}, \quad (\text{A.3})$$

To estimate n_c , two parameters must be assumed: n_d and L_d , while L_c is derived from observations as two times the core radius. The column density: $n_H = n_c L_c + n_d L_d$ removes one degree of freedom, giving

$$n_c = N_H \left(\frac{1 + \sqrt{1 - \left(\frac{L_d}{L_c} + 1 \right) \left(1 - \frac{\langle n_H \rangle_m L_d}{N_H} \right)}}{L_c + L_d} \right), \text{ if } L_d \text{ assumed.} \quad (\text{A.4})$$

$$\text{or } n_c = \frac{n_d}{2} \left(1 + \sqrt{1 - \frac{4N_H}{L_c n_d} \left(1 - \frac{\langle n_H \rangle_m}{n_d} \right)} \right), \text{ if } n_d \text{ assumed.} \quad (\text{A.5})$$

But these corrections can lead to $\langle n_H \rangle_m$ being higher than n_c , because the conditions $n_d > 0$ or $L_d > 0$ are not enforced. To rectify this, we clamp the core density as $n_c = \max(n_c, \langle n_H \rangle_m)$, with n_c computed from one of the previous formulas.

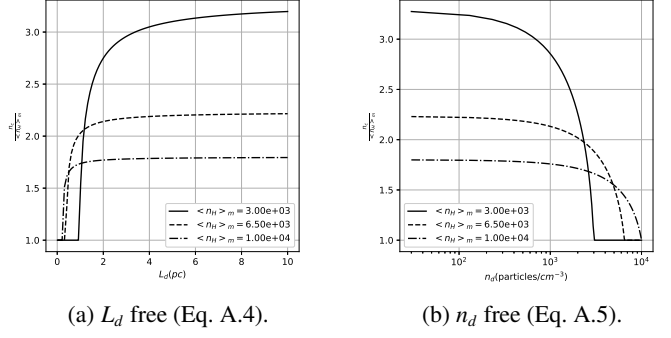


Fig. A.1: Correction $\langle n_H \rangle_m \rightarrow n_c$ for different $\langle n_H \rangle_m$; the vertical axis shows the ratio $n_c / \langle n_H \rangle_m$. The used parameters are $r_c = 0.05$ pc and $N_H = 10^{22}$ cm $^{-2}$.

If one wants to assume specific values for L_d and n_d :

$$n_c = \frac{\langle n_H \rangle_m}{2} \left(1 + \sqrt{1 + \frac{4L_d n_d}{L_c \langle n_H \rangle_m} \left(1 - \frac{n_d}{\langle n_H \rangle_m} \right)} \right) \quad (\text{A.6})$$

Figure A.1 displays the functions $n_c(L_d)$ (left) and $n_c(n_d)$ (right) for various values of $\langle n_H \rangle_m$. A notable feature is that n_c rapidly converges toward an upper limit when L_d becomes large or when n_d is small. The limits are

$$\lim_{L_d \rightarrow +\infty} n_c = \lim_{n_d \rightarrow 0} n_c = \sqrt{\frac{\langle n_H \rangle_m N_H}{L_c}}, \quad (\text{A.7})$$

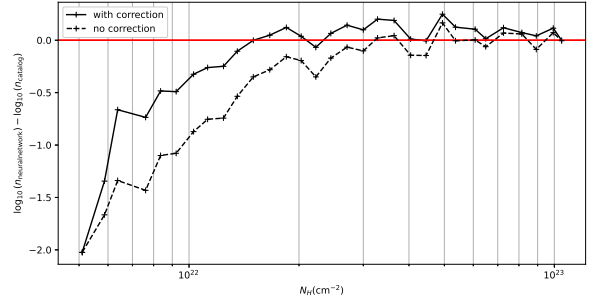


Fig. A.2: Differences in log space between the catalog core densities and the neural-network-predicted densities as a function of column density. The cloud is Orion B, neural network is UNet, and the dense-core catalog is from Könyves et al. (2020). The thick black line shows the density difference using the correction (Eq. A.4: $L_d = 10$ pc); the dotted black line shows the result without the correction. The red horizontal line indicates zero difference, i.e., where the neural-network prediction matches the catalog density.

We see in Figure A.2 that the correction indeed increases the dense-core density by an order of magnitude in diffuse regions, i.e., at low column densities. However, at high column densities, the two-medium approximation no longer holds, and therefore we can define an upper threshold, for example 3×10^{22} cm $^{-2}$, above which the density correction should not be applied.

Appendix B: Validation error and bias

Residuals distributions; Correction of the bias; Residuals error propagation to metrics (MC);

Appendix C: U-Net architecture variations

C.1. U-Net with Kolmogorov Arnold Networks

C.2. Size and Context embedding in U-Net

Appendix D: Score-based training set

Creating a training dataset is a crucial step in building an effective model. In the particular case of molecular clouds structures, we want to avoid overtraining on the borders of the simulation (WNM), as these regions contain less valuable information for our objective compared to the dense regions with filaments and cores. To achieve this, we implement an additional filtering step:

- We compute a "score" $s(\text{image})$ for each new candidate area before adding to the dataset. This score is calculated using a custom function that aligns with our training objectives.
- We then apply a filter function $f(s)$. If a randomly generated number r in the range $[0, 1]$ satisfies $r < f(s)$, the region is accepted and added to the training set. I.e $f(s)$ is the probability to accept and add an area with a score of s in the dataset.

This approach ensures that the dataset focuses more on important structures (filaments and cores) rather than sparse or less informative regions.

We can also use multiple scoring functions, each capturing different aspects of the dataset, and combine them using weighted (w_i) sums to account for various criteria. Mathematically, the final score s can be expressed as:

$$s = \sum_i s_i w_i \quad (\text{D.1})$$

This score can be made for the column density image $s(N_H)$ and the volume density image $s(\langle n_H \rangle_m)$.

D.1. Score functions

Smoothness score The score function used when generating our training set needs to be based on the smoothness of the selected area. This is because if an area is entirely within a diffuse region, it will exhibit low spatial frequency variation (i.e. only large-scale structures) and lack filaments.

To ensure that each selected image contains both diffuse regions and some filaments, we use the variance of the smoothness as a criterion. The smoothness itself is defined as the Laplacian of the image, which highlights areas with sharp density changes (such as filaments).

$$s_\Delta(x) = \text{Var}[\Delta(\log(1+x) - \min[\log(1+x)])], \quad (\text{D.2})$$

where x is the pixel value. The variance in equation D.2 is to get a mix of dense and high-density regions.

Difference score A basic criterion that can be chosen is the difference between the normalized column density and the normalized average volume density (Eq D.3). This helps identify regions where the local density distribution differs significantly between the two representations.

For example, when using mass-weighted average density, this criterion will highlight dense regions, since areas with high volume density but relatively lower column density indicate localized dense structures along the l.o.s.

$$s_{\text{ref}} = \text{Var}\left[\frac{N_H - \min(N_H)}{\max(N_H) - \min(N_H)} - \frac{\langle n_H \rangle_m - \min(\langle n_H \rangle_m)}{\max(\langle n_H \rangle_m) - \min(\langle n_H \rangle_m)}\right] \quad (\text{D.3})$$

D.2. Filter functions

As a filter function $f(s)$, we use a sigmoid:

$$f(s) = \frac{1}{1 + \exp(-A(s - B))}, \quad (\text{D.4})$$

where A is the step length and B the offset, and are chosen empirically.

D.3. Network robustness to training-set variations

Here, we test the effect of excluding WNM or large homogeneous regions from the training set for some of our architectures. We compare the model performances on two different datasets:

- One dataset is the same as used in this paper, containing approximately 100 regions of the ORION simulation, excluding WNM and favoring heterogeneous regions in the selection.
- The other dataset contains 170 regions, including all WNM regions from the simulation, with no selection bias toward heterogeneity.

U-Net test

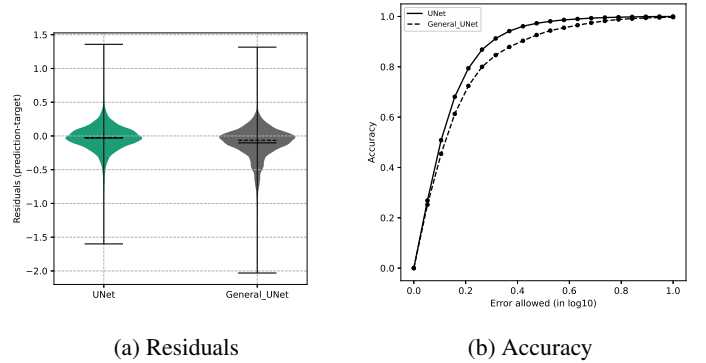


Fig. D.1

ciNN train a ciNN with less data

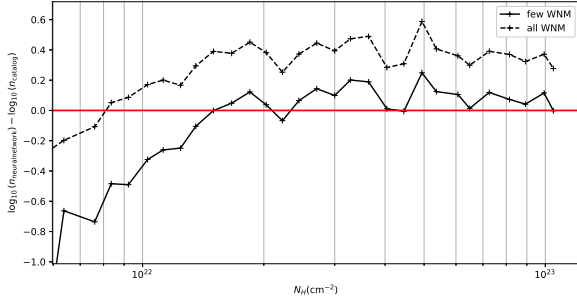


Fig. D.2: Differences in log space between the catalog core densities and the neural-network-predicted densities as a function of column density. Cloud is Orion B, model is UNet, and the dense-core catalog is from Könyves et al. (2020). Thick black line: dataset without WNM; dotted black line: dataset with WNM. The red horizontal line indicates zero difference.

Appendix E: Examples of dense core predictions

Figure: horizontal: Powerlaw, Unet, cINN, DDPM vertical: top: column density, bot: average volume density.
One or few for differents environments: low column density, high column density, in filaments, outside filaments...